Київський національний університет
Імені Тараса Шевченка.
Кафедра: Мережевих та інтернет-технологій.

Лабораторна робота №2

з дисципліни: Системне Програмування
На тему: “Паралелізм даних та паралелізм задач”

Студента 4 курсу:
Групи МІТ-41
Мулико Володимира

Київ - 2025р.

Завдання 1:

Реалізуйте програму на C#, яка створює дві задачі (Task) для паралельного виконання:

- перша задача виводить числа від 1 до 10 з паузою 200 мс;
- друга задача виводить літери від A до J з паузою 200 мс.

Використайте методи Start() та WaitAll().

Питання для звіту: поясніть різницю між класами Thread та Task.

Код:

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
   static void Main(string[] args)
   {
       Task task1 = new Task(() =>
       {
           for (int i = 1; i <= 10; i++)
           {
               Console.Write($"{i} ");
               Thread.Sleep(200);
           }
       });

       Task task2 = new Task(() =>
       {
           for (char c = 'A'; c <= 'J'; c++)
           {
               Console.Write($"{c} ");
               Thread.Sleep(200);
           }
       });

       task1.Start();
       task2.Start();

       Task.WaitAll(task1, task2);

       Console.WriteLine("\nОбидві задачі завершено.");
   }
```

```
}
```

У цьому коді створюються два об'єкти класу Task, кожен із яких зациклено виводить послідовність (числа або літери) з імітацією затримки через Thread.Sleep(200), після чого вони запускаються вручну методом Start(), а головний потік програми очікує їхнього завершення за допомогою Task.WaitAll(task1, task2). Основна різниця між Thread та Task полягає в тому, що Thread є низькорівневим потоком операційної системи з великими витратами ресурсів на створення, тоді як Task — це високорівнева абстракція над пулом потоків (ThreadPool), яка дозволяє ефективніше керувати ресурсами, легше повертати результати виконання та підтримує сучасний синтаксис async/await.

Вивід:

```
● → TASK1 dotnet run
  A 1 B 2 C 3 4 D 5 E 6 F 7 G 8 H 9 I 10 J
  Обидві задачі завершено.
```

Завдання 2:

Створіть програму, яка запускає три задачі, кожна з яких виконує підрахунок до 5.

Виведіть у консоль ідентифікатор (Id) кожної задачі та поточний ідентифікатор (CurrentId) під час виконання.

Питання для звіту: чи є ідентифікатори задач впорядкованими? Як їх можна використати для відстеження виконання?

Код:

```
using System;
using System.Threading.Tasks;

class Program
{
   static void Main(string[] args)
   {
       Task[] tasks = new Task[3];

       for (int i = 0; i < 3; i++)
       {
           tasks[i] = Task.Run(() =>
```

```
            {
                for (int j = 1; j <= 5; j++)
                {
                    Console.WriteLine($"Task Id: {Task.CurrentId} рахує:
{j}");
                }
            });
            Console.WriteLine($"Запущено задачу з Id: {tasks[i].Id}");
        }

        Task.WaitAll(tasks);
    }
}
```

Програма використовує цикл для створення та негайного запуску трьох задач через Task.Run, де всередині кожної задачі за допомогою властивості Task.CurrentId виводиться її унікальний номер під час ітерацій від 1 до 5. Ідентифікатори задач не є суворо впорядкованими за часом виконання, оскільки вони призначаються системою в міру створення об'єктів і залежать від внутрішньої логіки планувальника задач .NET, тому їх не можна використовувати для визначення черговості завершення. Для відстеження виконання ці ідентифікатори корисні при логуванні або налагодженні програми, оскільки вони дозволяють чітко розрізняти, яка саме паралельна задача вивела конкретне повідомлення в консоль або спричинила помилку в багатопотоковому середовищі.

Вивід:

```
➜  TASK2 dotnet run
Запущено задачу з Id: 1
Task Id: 1 рахує: 1
Task Id: 1 рахує: 2
Task Id: 1 рахує: 3
Task Id: 1 рахує: 4
Task Id: 1 рахує: 5
Запущено задачу з Id: 2
Запущено задачу з Id: 3
Task Id: 3 рахує: 1
Task Id: 3 рахує: 2
Task Id: 3 рахує: 3
Task Id: 3 рахує: 4
Task Id: 3 рахує: 5
Task Id: 2 рахує: 1
Task Id: 2 рахує: 2
Task Id: 2 рахує: 3
Task Id: 2 рахує: 4
Task Id: 2 рахує: 5
```

Завдання 3:

Реалізуйте програму, яка:

- створює задачу для обчислення суми чисел від 1 до N (N вводиться користувачем).
- використовує метод ContinueWith() для запуску другої задачі, яка виводить результат у консоль.
- використовує лямбда-вираз для реалізації продовження.

Питання для звіту: які переваги має використання продовжень задач у порівнянні з класичним підходом синхронізації?

Код:

```
using System;
using System.Threading.Tasks;

class Program
{
   static void Main(string[] args)
   {
       Console.Write("Введіть число N: ");
       int n = int.Parse(Console.ReadLine());

       Task<int> sumTask = Task.Run(() =>
       {
           int sum = 0;
           for (int i = 1; i <= n; i++)
           {
               sum += i;
           }
           return sum;
       });

       sumTask.ContinueWith(t =>
       {
                Console.WriteLine($"Сума чисел від 1 до {n} дорівнює:
{t.Result}");
       });

       sumTask.Wait();
   }
}
```

Програма зчитує число N від користувача та запускає задачу для обчислення суми, результат якої передається в наступну задачу через метод ContinueWith, де за допомогою лямбда-виразу та властивості t.Result отримане значення виводиться на екран. Використання продовжень задач має значну перевагу над класичною синхронізацією (як-от блокування через Wait() або lock), оскільки воно дозволяє будувати неблокуючі ланцюжки операцій, де ресурси потоку вивільняються під час очікування завершення попереднього етапу, що суттєво підвищує масштабованість та відгук додатку. Це дозволяє уникнути простою процесора та створювати складні асинхронні сценарії, де кожна наступна дія виконується автоматично лише після успішного завершення попередньої, не зупиняючи при цьому роботу всього додатка.

Вивід:

```
● → TASK3 dotnet run
  Введіть число N: 10
  Сума чисел від 1 до 10 = 55
```

Завдання 4:

Розробіть програму, яка виконує три методи паралельно за допомогою Parallel.Invoke():

- метод 1: обчислює факторіал числа.
- метод 2: обчислює суму чисел від 1 до N.
- метод 3: виводить повідомлення у консоль з паузою 300 мс.

Питання для звіту: які переваги має використання Parallel.Invoke() у порівнянні зі створенням окремих задач?

Код:

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
   static void Main(string[] args)
   {
       int n = 10;

       Parallel.Invoke(
           () => {
               long factorial = 1;
```

```
                for (int i = 1; i <= 5; i++) factorial *= i;
                Console.WriteLine($"Факторіал 5: {factorial}");
            },
            () => {
                int sum = 0;
                for (int i = 1; i <= n; i++) sum += i;
                Console.WriteLine($"Сума до {n}: {sum}");
            },
            () => {
                Thread.Sleep(300);
                Console.WriteLine("Повідомлення після паузи 300 мс");
            }
        );

        Console.WriteLine("Всі паралельні методи завершені.");
    }
}
```

Програма використовує метод Parallel.Invoke для одночасного запуску трьох різних операцій (обчислення факторіала, суми та виводу повідомлення), що дозволяє автоматизувати розподіл навантаження між ядрами процесора без необхідності вручну керувати кожною окремою задачею. Основна перевага Parallel.Invoke() у порівнянні зі створенням окремих об'єктів Task полягає в лаконічності коду та автоматичній синхронізації, оскільки цей метод блокує виконання головного потоку до тих пір, поки всі передані дії не будуть повністю завершені, позбавляючи програміста необхідності використовувати WaitAll() або інші складні механізми очікування. Це робить код значно чистішим і надійнішим для сценаріїв, де потрібно просто виконати кілька незалежних дій паралельно і дочекатися їх спільного результату перед тим, як рухатися далі.

Вивід:

```
→ TASK4 dotnet run
Сума до 10: 55
Факторіал 5: 120
Повідомлення після паузи 300 мс
Всі паралельні методи завершені.
```

Висновки:

Під час виконання лабораторної роботи я опанував практичні навички роботи з бібліотекою Task Parallel Library (TPL) у мові програмування C#. Було реалізовано різні підходи до організації паралельних обчислень, зокрема створення об’єктів класу Task з ручним запуском через Start(), використання фабричного методу Task.Run() та організацію паралельного виконання незалежних методів за допомогою Parallel.Invoke(). Особливу увагу було приділено механізмам синхронізації та побудові асинхронних ланцюжків за допомогою методу ContinueWith(), що дозволяє ефективно передавати результати між задачами без блокування головного потоку.

Аналіз отриманих результатів підтвердив, що використання класу Task є продуктивнішим рішенням порівняно з низькорівневими потоками Thread, оскільки задачі автоматично керуються пулом потоків, що оптимізує споживання ресурсів процесора та пам'яті. Робота з ідентифікаторами Id та CurrentId продемонструвала асинхронну природу виконання задач, де порядок завершення не завжди збігається з порядком запуску. Отримані знання є фундаментальними для розробки високонавантажених систем та сучасних додатків із чутливим інтерфейсом користувача.